

Práctico 3

Procesos, threads y concurrencia

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

Resumen

Este documento presenta la resolución integral del Práctico 3 de la materia Sistemas Operativos (UNRC, 2026), centrado en la administración de *procesos* y *threads*, los mecanismos de *planificación* de CPU y los problemas de *concurrencia* y *sincronización*. Cada ejercicio se justifica con la teoría de los capítulos *Procesos y threads* y *Concurrencia e IPC* de las *Notas del curso* (Marcelo Arroyo) y se acompaña con comandos, salidas, fragmentos de código y experimentos verificados sobre GNU/Linux y macOS. La discusión recorre los estados de un proceso, el control con señales, el árbol de procesos, la planificación con prioridades, las condiciones de carrera, la sincronización con `flock`, mutexes POSIX, monitores Java, las operaciones del kernel ante interrupciones de timer y de I/O, la prueba semi-formal del algoritmo de Peterson, el problema productor-consumidor con semáforos POSIX (con threads y con procesos), el análisis de *deadlocks* en presencia de interrupciones y de grafos de recursos, y el productor-consumidor con *colas de mensajes POSIX* controladas con SIGSTOP/SIGCONT.

Índice

1. Introducción	2
2. Ejercicio 1 — Estado de los procesos con <code>ps</code>	2
3. Ejercicio 2 — Control de un proceso con señales	3
4. Ejercicio 3 — Árbol de procesos con <code>pstree</code>	5
5. Ejercicio 4 — Planificación con <code>nice</code> y <code>renice</code>	6
6. Ejercicio 5 — <code>counter.c</code> , race condition y <code>flock</code>	8
7. Ejercicio 6 — <code>pthread-example.c</code> , mutex y stacks	10
8. Ejercicio 7 — Multithreading en Java	12
9. Ejercicio 8 — Operaciones del kernel ante interrupción del timer	15
10. Ejercicio 9 — Operaciones del kernel ante read sobre disco	16
11. Ejercicio 10 — Demostración semi-formal de Peterson	18
12. Ejercicio 11 — Productor-consumidor con semáforos POSIX	20
13. Ejercicio 12 — Deadlock entre <code>interrupt_handler</code> y <code>f()</code>	25
14. Ejercicio 13 — Grafo de recursos y deadlock con liberación temprana	27
15. Ejercicio 14 — Productor-consumidor con <code>mq</code> POSIX y señales	29

1. Introducción

Un *proceso* es una instancia de un programa en ejecución; un *thread* (o *tarea*) es la unidad mínima planificable por el kernel. Un proceso tiene siempre al menos un thread, su *main thread*, y puede crear threads adicionales que comparten el espacio de memoria del proceso pero tienen pilas independientes. Los recursos asignados a un proceso —memoria, descriptores de archivos, manejadores de señales, áreas de memoria compartida— son administrados explícitamente por el kernel mediante una *tabla de procesos*.

La teoría del curso modela el ciclo de vida de un proceso con cinco estados: **NEW**, **RUNNING**, **RUNNABLE** (o **READY**), **WAITING** (o **SLEEPING**), **ZOMBIE** y **TERMINATED**. Las transiciones se disparan por *interrupciones*, *traps* y *llamadas al sistema*: el timer fuerza la transición **RUNNING** → **RUNNABLE** al consumir el quantum; un **read** sobre disco produce **RUNNING** → **SLEEPING**; un **wakeup** produce **SLEEPING** → **RUNNABLE**; **exit** lleva al proceso a **ZOMBIE** hasta que el padre haga **wait**.

Cuando varias tareas comparten un recurso aparece la *conurrencia*. La porción de código que accede al recurso compartido se llama *región crítica* y la dependencia del resultado en el orden de ejecución se llama *condición de carrera* (*race condition*). Para evitar trazas no deseadas el SO ofrece una jerarquía de *mecanismos de sincronización*: spinlocks, sleep locks, semáforos, variables de condición, monitores y mensajes. Cada uno apunta a un escenario distinto: regiones cortas vs. largas, monoprocesador vs. multiprocesador, memoria compartida vs. procesos aislados.

El práctico ejercita progresivamente todos estos conceptos: parte del *observable* (estados de los procesos visibles con **ps**, control con señales, árbol con **pstree**), pasa por la *planificación con prioridades* (**nice/renice**), introduce las *race conditions* (**counter.c** con dos procesos, **pthread-example.c** con dos hilos) y sus correcciones (**flock**, **pthread_mutex_t**, **synchronized** de Java), describe el comportamiento del kernel en dos escenarios típicos (timer y **read** con I/O), demuestra semi-formalmente el algoritmo de Peterson y termina con el clásico problema del *productor-consumidor* implementado con semáforos POSIX, en sus dos variantes: con threads (memoria compartida) y con procesos (named semaphores y shared memory).

2. Ejercicio 1 — Estado de los procesos con ps

Se pide ejecutar **ps aux**, identificar los estados de los procesos, ordenar por uso de CPU y de memoria, y listar los procesos del usuario actual.

1.a Lectura de la salida y estados

```
ps aux | head -n 5
```

Cada fila representa una tarea. Las columnas claves son: **USER** y **PID** (identificación), **%CPU**/**%MEM** (consumo instantáneo), **VSZ**/**RSS** (memoria virtual y residente), **TTY** (terminal controladora), **STAT** (estado codificado), **TIME** (CPU acumulado) y **COMMAND** (línea de comando).

La columna **STAT** expone los estados teóricos del modelo de tareas. La correspondencia entre los códigos **ps** y los estados del PCB es:

Estado teórico	Linux	macOS	Significado en el kernel
RUNNING/RUNNABLE	R	R	En la CPU o en la cola de <i>ready</i>
SLEEPING interrumpible	S	S/I	En <i>wait queue</i> ; despertable por señal
SLEEPING no interrumpible	D	U	I/O síncrono; no despertable por señal
Detenido por señal	T	T	SIGSTOP/SIGTSTP
ZOMBIE	Z	Z	Terminó pero el padre no hizo <code>wait()</code>

El estado `TERMINATED` no aparece en `ps` porque, una vez hecho el `wait()`, el descriptor se libera y el proceso desaparece de la tabla. Los flags adicionales (`<`, `N`, `s`, `+`, `l`, `L`) refinan la información: prioridad no estándar, líder de sesión, foreground, multi-hilo, páginas *locked* en RAM.

1.b Ordenar por CPU y memoria

En Linux:

```
ps aux --sort=-%cpu | head
ps aux --sort=-%mem | head
```

En macOS, `ps` hereda de BSD y no soporta `-sort`. El equivalente es:

```
ps aux -r | head -n 10 # por CPU descendente
ps aux -m | head -n 10 # por memoria residente descendente
```

1.c Procesos del usuario actual

```
ps -U "$USER" -o user,pid,%cpu,%mem,stat,command
```

Alternativamente, filtrando con `awk`:

```
ps aux | awk -v u="$USER" '$1 == u'
```

Conexión con la teoría

`ps` no es un comando autónomo: lee directamente las estructuras del kernel que la teoría llama *task descriptors*. En Linux la información vive bajo `/proc/<PID>/` (un *pseudo-filesystem* expuesto por el kernel) y `ps` la formatea para el usuario. Cada columna corresponde a un campo del descriptor: `PID` → identificador, `STAT` → campo `state`, `%CPU/TIME` → contadores acumulados por el scheduler, `VSZ/RSS` → *mapa de memoria* del proceso, `COMMAND` → programa del que es instancia.

3. Ejercicio 2 — Control de un proceso con señales

Se pide manipular el estado del script `runseconds.sh` (un `busy-wait` de 120 segundos) lanzándolo en background, deteniéndolo con `SIGSTOP`, reanudándolo con `SIGCONT` y finalmente esperando o matándolo.

2.a El script

```
#!/bin/bash
# Run for 120 seconds
end=$((SECONDS + 120))
```

```
while [ $SECONDS -lt $end ]; do
    : # this command does nothing
done

echo "Done!"
```

La variable `SECONDS` es interna del shell y avanza solo cuando el shell se está ejecutando: si el proceso está en estado T (detenido), `SECONDS` no avanza y la duración total se mide en tiempo de CPU corrido, no en tiempo real.

2.b Lanzar en background y verificar R

```
chmod +x runseconds.sh
./runseconds.sh &
PID=$!
ps -o pid,stat,command -p $PID
```

```
PID STAT CMD
12345 R bash ./runseconds.sh
```

2.c Pasar a T con SIGSTOP

```
kill -STOP $PID
ps -o pid,stat,command -p $PID
```

```
PID STAT COMMAND
12345 T bash ./runseconds.sh
```

`SIGSTOP` (señal 19 en Linux) no puede ser ignorada ni capturada; el kernel saca al proceso de toda cola de planificación hasta recibir `SIGCONT`.

2.d Reanudar con SIGCONT

```
kill -CONT $PID
ps -o pid,stat,command -p $PID
```

El estado vuelve a R.

2.e Esperar o terminar

```
wait $PID # esperar a que termine solo
kill $PID # SIGTERM (terminacion ordenada)
kill -KILL $PID # SIGKILL (no se puede ignorar)
```

Conexión con la teoría

Cada paso del ejercicio corresponde a una transición concreta del diagrama de estados (Figura 5 de las Notas 5–6):

Paso	Señal	Transición en el modelo teórico
2.b	(lanzamiento)	NEW → RUNNABLE → RUNNING
2.c	SIGSTOP	RUNNING → <i>Stopped</i> (fuera del scheduler)
2.d	SIGCONT	<i>Stopped</i> → RUNNABLE → RUNNING
2.e	SIGTERM/fin	RUNNING → ZOMBIE → TERMINATED

SIGSTOP es el ejemplo concreto del estado *controlado por el SO*: el proceso no espera I/O (no está SLEEPING) ni compite por CPU (no está RUNNABLE); fue retirado de toda cola hasta recibir SIGCONT. SIGTERM puede ser capturada por el proceso para hacer una salida ordenada; SIGKILL y SIGSTOP las maneja el kernel directamente y no pueden ser interceptadas, lo que las convierte en herramientas seguras contra procesos colgados.

4. Ejercicio 3 — Árbol de procesos con pstree

Se pide visualizar el árbol de procesos del sistema y determinar el primer proceso lanzado y su PID.

3.a Salida de pstree

En GNU/Linux **pstree** viene preinstalado como parte de *psmisc*. En macOS hace falta instalar la versión de Fred Hucht con Homebrew (**brew install pstree**); las opciones difieren ligeramente de las de Linux.

```
pstree
```

Las primeras líneas son aproximadamente:

```
--= 00001 root /sbin/launchd (macOS)
|--- 00312 root /usr/libexec/logd
|--- 00313 root /usr/libexec/smd
...
```

Los conectores `--=` y `|--=` arman la jerarquía y los números de cinco dígitos son los PIDs.

3.b El primer proceso

El primer proceso de espacio de usuario tiene siempre **PID 1** y es ancestro de todos los demás:

```
ps -p 1 -o pid,ppid,user,comm
```

```
PID PPID USER COMM
1 0 root /sbin/launchd
```

El PPID = 0 corresponde al proceso interno del kernel (*scheduler/swapper*) que no aparece en espacio de usuario. Quién ejecuta como PID 1 depende del sistema: **launchd** en macOS, **systemd** en Linux moderno, **init** (SysV/BusyBox) en Linux tradicional, **init** en FreeBSD.

Conexión con la teoría

La Figura 6 del capítulo *Procesos y threads* describe la secuencia de *bootstrapping* en sistemas tipo UNIX:

```

initcode (pid=1, hardcoded en el kernel)
| exec("init")
init (pid=1)
| fork(); child: exec("tty")
tty (pid=2)
| exec("login")
login (pid=3)
| exec("sh")
shell (pid=4)

```

El kernel construye `initcode` en una página especial al terminar el boot. Ese código invoca `exec("init")`, que reemplaza la imagen del proceso PID 1 por el binario del `init` del sistema. Toda otra tarea de espacio de usuario surge por `fork()` desde algún descendiente de PID 1: por construcción, `ps tree` produce un árbol y nunca un bosque.

Esta arquitectura justifica la *adopción de huérfanos* por parte de PID 1: si un proceso muriese sin hacer `wait()` sobre sus hijos, estos quedarían en estado ZOMBIE indefinidamente. Reasignando los huérfanos a `init`, el sistema asegura que tarde o temprano alguien hará `wait()` y la tabla de procesos se mantenga limpia.

5. Ejercicio 4 — Planificación con `nice` y `renice`

Se pide ejecutar un proceso CPU-bound, lanzar una segunda instancia con menor prioridad y observar las diferencias en *context switches* con `cat /proc/<PID>/sched | grep nr_switches`. Después, modificar la prioridad en caliente y, finalmente, lanzar un proceso con prioridad superior.

4.a Programa de carga

```

#!/bin/bash
# busyloop.sh: ciclo infinito, 100% CPU.
while ;; do ;; done

```

4.b Una instancia, observada con `top`

```

./busyloop.sh
# en otra terminal:
top # tecla P ordena por %CPU; q sale

```

`bash busyloop.sh` consume cerca del 100% de un núcleo, con NI=0 y PR por defecto.

4.c Segunda instancia con `nice +15`

```

nice -n 15 bash ./busyloop.sh

```

`top` muestra las dos instancias compitiendo: la nueva con NI=15 y PR numéricamente más alto (recordar que en Linux un PR mayor significa *menos* prioridad).

Cambios de contexto. El kernel expone los context switches acumulados de cada proceso en `/proc/<PID>/sched`:

```
cat /proc/<PID>/sched | grep nr_switches
```

¿El proceso de menor prioridad ejecuta más o menos cambios de contexto por unidad de tiempo? Más. El razonamiento usa la regla central del CFS (*Completely Fair Scheduler*) de Linux: el *quantum* asignado a una tarea es proporcional a su *peso*, y el peso es decreciente con *nice*. Concretamente, el peso aproximado es $1024 \cdot 1.25^{-\text{nice}}$, de modo que cada incremento unitario de *nice* reduce el peso en $\sim 20\%$. Un proceso con *nice*=15 tiene un peso aproximadamente 15 veces menor que uno con *nice*=0, por lo que:

- recibe *time slices* más cortos $\rightarrow$ es desalojado más seguido $\rightarrow$ *nr_switches* crece más rápido por unidad de tiempo (paradójicamente, el *menos prioritario* hace *más* context switches);
- pero la cantidad **total** de CPU que recibe es menor.

Para verlo en vivo:

```
watch -n1 "cat /proc/<PID_NICE15>/sched | grep nr_switches"
```

4.d Cambiar prioridad en caliente con *renice*

```
renice -n 5 -p <PID> # subir nice (bajar prioridad)
renice -n -5 -p <PID> # bajar nice (subir prioridad, requiere root)
ps -o pid,ni,pri,comm -p <PID>
```

4.e Mayor prioridad y por qué pide *sudo*

```
sudo nice -n -10 bash ./busyloop.sh
```

Bajar *nice* a un valor negativo aumenta la prioridad por encima de la del resto de los procesos del usuario y, por eso, requiere **root**. La razón es de *seguridad y equidad*: si cualquier usuario pudiera mejorarse a sí mismo la prioridad, podría desplazar a procesos del sistema y monopolizar la CPU, transformando una limitación del scheduler en una herramienta de *denegación de servicio*. Por eso la convención histórica de UNIX es que solo **root** puede asignar *nice* negativos. Un usuario sin privilegios sí puede *subir* el *nice* de un proceso suyo (cederle prioridad a otros), nunca lo opuesto. Esto refleja la regla general: *cada usuario puede ceder sus propios recursos pero no apropiarse de los ajenos*.

Conexión con la teoría

El *quantum* es “el período de tiempo máximo de uso de la CPU” (Notas 5–6, *Cambios de contexto*). La interrupción del *timer* es la implementación concreta del *preemptive multitasking*: gracias a ella el kernel puede recuperar la CPU sin la cooperación del proceso, evitando que un thread que no invoca *yield()* voluntariamente monopolice el procesador. Cada desalojo es un *context switch involuntario* contabilizado en *nr_switches*.

Limpieza.

```
kill -f busyloop.sh
```

6. Ejercicio 5 — counter.c, race condition y flock

Se pide compilar `counter.c`, observar que ejecuciones secuenciales producen 2000 pero ejecuciones concurrentes pierden incrementos, explicar la causa y solucionarlo con `flock()`. Por último, justificar si `fd` es una variable compartida entre las distintas instancias.

5.a El programa

```
1 #include <fcntl.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <unistd.h>
5 #include <stdio.h>
6 #include <sys/file.h>
7
8 #define N 10
9
10 int main()
11 {
12     int pid = getpid(), i;
13     int fd = open("counter.dat", O_RDWR);
14     (void)pid;
15
16     for (i=0; i<1000; i++) {
17         char s[N];
18         int n;
19
20         lseek(fd, 0, SEEK_SET);
21         read(fd, s, N);
22         n = atoi(s);
23         sprintf(s, "%d", n+1);
24         lseek(fd, 0, SEEK_SET); // rewind
25         write(fd, s, strlen(s));
26         fsync(fd);
27     }
28     close(fd);
29     return 0;
30 }
```

```
gcc -Wall -Wextra -o counter counter.c
echo 0 > counter.dat
```

5.b Ejecución secuencial

```
echo 0 > counter.dat
./counter ; ./counter
cat counter.dat # 2000
```

Cada ejecución empieza después de que la anterior cerró el archivo, así que ningún incremento se pierde.

5.c Ejecución concurrente sin lock

```
echo 0 > counter.dat
./counter & ./counter
wait
```



```
cat counter.dat # < 2000, no determinista
```

El resultado es menor a 2000 y cambia entre corridas: 1826, 1740, 1766, etc.

Por qué se pierden incrementos. La secuencia *leer-modificar-escribir* no es atómica. Con dos procesos *A* y *B* en paralelo, una traza típica es:

```
A lseek(0); read -> "100" (v=100)
                        B lseek(0); read -> "100" (v=100)
A sprintf("101"); write 3 bytes
                        B sprintf("101"); write 3 bytes
                        <-- pisa con el mismo valor
```

Es el patrón clásico de *lost update*: ambos procesos leen el mismo valor, ambos escriben el mismo resultado, y un incremento se pierde silenciosamente. `fsync()` solo garantiza persistencia en disco; no excluye a otro proceso que esté queriendo escribir.

5.d Solución con flock()

```
1 for (i=0; i<1000; i++) {
2     char s[N];
3     int n;
4
5     flock(fd, LOCK_EX);
6
7     lseek(fd, 0, SEEK_SET);
8     read(fd, s, N);
9     n = atoi(s);
10    sprintf(s, "%d", n+1);
11    lseek(fd, 0, SEEK_SET);
12    write(fd, s, strlen(s));
13    fsync(fd);
14
15    flock(fd, LOCK_UN);
16 }
```

```
echo 0 > counter.dat
./counter_locked & ./counter_locked
wait
cat counter.dat # 2000, deterministico
```

`flock(fd, LOCK_EX)` toma un lock exclusivo *advisory* sobre el descriptor. Mientras un proceso lo retiene, cualquier otro que intente `LOCK_EX` se duerme hasta que el primero llame `LOCK_UN`. La región crítica queda lo más corta posible: solo el read-modify-write, no todo el bucle, lo que preserva el paralelismo cuando la región no se está ejecutando.

5.e ¿Es fd una variable compartida?

No. Cuando dos procesos hacen `./counter` por separado, cada uno corre en su propio espacio de direcciones y cada uno hace su propio `open("counter.dat", O_RDWR)`. El kernel les devuelve un descriptor en la *tabla de descriptores de cada proceso*, que casualmente probablemente sea 3 en ambos (porque 0, 1, 2 ya están ocupados por `stdin/stdout/stderr`), pero son entradas en tablas distintas. Apuntan al mismo *inodo* del filesystem, no a la misma `struct file` del kernel; cada uno tiene su propio offset.

Esto explica por qué la race se da: como los offsets son independientes, ambos procesos pueden hacer `lseek(0)` y `read` sin que el kernel los serialice. El recurso compartido entre las dos instancias **no es fd**, sino el **archivo en disco**, y la sincronización tiene que hacerse sobre ese archivo (con `flock`).

Conexión con la teoría

Las Notas 5-6 (*Concurrencia e IPC*) definen la *condición de carrera* como “el comportamiento del sistema que depende de la secuencia de la ocurrencia de eventos externos que pueden generar flujos de ejecución no deseados”. En este ejercicio la *región crítica* es el bloque `lseek + read + atoi + sprintf + lseek + write`, el *recurso compartido* es `counter.dat`, y el *interleaving no deseado* es el *lost update* mostrado arriba.

`flock()` es la implementación concreta del patrón *acquire-uso-release* (Listado 1 del capítulo *Locks*):

```
1 acquire(&l); // flock(fd, LOCK_EX)
2 use_resource(r); // operacion sobre el archivo
3 release(&l); // flock(fd, LOCK_UN)
```

provee *exclusión mutua* sobre la región crítica.

7. Ejercicio 6 — pthreads-example.c, mutex y stacks

Se pide ejecutar el programa, observar que falla, corregirlo con `pthread_mutex_t`, mostrar la dirección de la variable local `tid` en cada hilo y verificar experimentalmente que todos los hilos ejecutan en el mismo espacio de memoria del proceso.

6.a Programa original y bug

```
1 int counter = 0; // Shared resource
2
3 void* increment(void* thread_id) {
4     int tid = *((int *) thread_id);
5     for (int i = 0; i < 100000; i++) {
6         if (i == 0) {
7             printf("Thread %d running...\n", tid);
8             printf("Address of local variable: %p\n", &tid);
9         }
10        counter++;
11    }
12    return NULL;
13 }
14
15 int main() {
16     pthread_t t1, t2;
17     int tn1 = 1, tn2 = 2;
18
19     pthread_create(&t1, NULL, increment, &tn1);
20     pthread_create(&t2, NULL, increment, &tn2);
21
22     pthread_join(t1, NULL);
23     pthread_join(t2, NULL);
24
25     printf("Final Counter Value: %d\n", counter);
26     return 0;
27 }
```

```
gcc -Wall -Wextra -pthread -o pthreads-example pthreads-example.c
./pthreads-example
```

Salida (ejemplo):

```
Thread 1 running...
Address of local variable: 0x16fcdefb4
Thread 2 running...
Address of local variable: 0x16fd6afb4
Final Counter Value: 102483
```

El valor final es menor a 200000. La causa es que `counter++` no es atómico: en máquina se traduce aproximadamente a tres instrucciones *load-modify-store*:

```
lw t0, counter
addi t0, t0, 1
sw t0, counter
```

Dos hilos pueden leer el mismo valor antes de que cualquiera escriba; el último `store` pisa al otro. Como ambos hilos viven en el mismo proceso, comparten la misma página de memoria que aloja `counter`: no hay copias separadas, hay una única palabra de memoria.

6.b Corrección con `pthread_mutex_t`

```
1 pthread_mutex_t mtx = PTHREAD_MUTEX_INITIALIZER;
2
3 for (int i = 0; i < 100000; i++) {
4     if (i == 0) {
5         printf("Thread %d running...\n", tid);
6         printf("Address of local variable: %p\n", &tid);
7     }
8     pthread_mutex_lock(&mtx);
9     counter++;
10    pthread_mutex_unlock(&mtx);
11 }
```

```
Thread 1 running...
Address of local variable: 0x16f702fb4
Thread 2 running...
Address of local variable: 0x16f78efb4
Final Counter Value: 200000
```

Siempre 200000, de manera determinista. El mutex implementa un *sleep lock*: si un hilo intenta tomar un lock ocupado, el kernel lo pone en `SLEEPING` hasta que el otro hilo lo libere. Esto contrasta con un *spinlock*, que haría busy-wait.

6.c Direcciones de `tid` y distancia entre stacks

```
Thread 2 running...
Address of local variable: 0x16fb26fb4
Thread 1 running...
Address of local variable: 0x16fa9afb4
```

```
|&tid_2 - &tid_1| = |0x16fb26fb4 - 0x16fa9afb4| = 0x8c000  
~ 573 440 bytes ~ 560 KB
```

El valor exacto varía entre ejecuciones (ASLR del kernel), pero la *distancia* entre stacks consecutivos es constante y corresponde al **tamaño del stack reservado por hilo**:

- macOS (Darwin): ~ 512 KB para hilos no-main.
- Linux/glibc: 8 MB (0x800000, equivalente a `ulimit -s = 8192 KB`).
- FreeBSD: 2 MB.

¿Están en la misma pila? No. Cada hilo tiene su propio stack, asignado por la biblioteca de *pthread*s con `mmap` privado al momento de `pthread_create`. La distancia observada confirma que son stacks separados.

6.d Experimento: todos los hilos comparten el address space

Para verlo de manera directa, basta con que cada hilo imprima la dirección del global `counter`, su PID, y la dirección de su `tid` local:

```
1 printf("[T%d] pid=%d, &counter=%p, &tid=%p\n",  
2      tid, getpid(), (void*)&counter, (void*)&tid);
```

```
[T1] pid=44213, &counter=0x10054af80, &tid=0x16fa9afb4  
[T2] pid=44213, &counter=0x10054af80, &tid=0x16fb26fb4
```

Lo observado:

- **Mismo PID** → son tareas dentro del mismo proceso.
- **Misma dirección para &counter** → la variable global vive en la misma página virtual; ambos hilos usan la misma tabla de páginas. *Por eso* el `counter++` sin lock genera la race condition.
- **&tid distintos** → stacks independientes.

Conexión con la teoría

El experimento es la confirmación empírica del modelo de proceso multithreaded de las Notas 5–6 (Figura 1): “cada thread usa su propia pila aunque todos los threads comparten el espacio de memoria del proceso”. Texto, datos, heap y variables globales como `counter` son **compartidos** entre todos los hilos. El stack es **privado** a cada hilo. La elección entre spinlock y sleep lock (mutex) sigue la regla de la teoría: spinlock para regiones cortas, mutex para regiones largas o con alta contención.

8. Ejercicio 7 — Multithreading en Java

Se pide analizar (1) los mecanismos de creación y control de threads en Java, (2) los mecanismos de sincronización y su clasificación, y (3) implementar el equivalente del Ejercicio 6 sin race conditions.

7.a Creación y control de threads

Java provee soporte de hilos de *primera clase* en la JVM. Cada hilo Java se corresponde uno-a-uno con un hilo nativo del sistema operativo. Hay dos formas equivalentes de crearlos:

```

1 // Forma 1: extender Thread (poco flexible)
2 class MiHilo extends Thread {
3     @Override public void run() { /* ... */ }
4 }
5 new MiHilo().start();
6
7 // Forma 2: implementar Runnable (preferida)
8 class Tarea implements Runnable {
9     @Override public void run() { /* ... */ }
10 }
11 new Thread(new Tarea()).start();
12 new Thread(() -> { /* ... */ }).start(); // con lambda

```

Métodos clave:

Método	Qué hace
<code>start()</code>	Pide a la JVM crear el hilo y ejecutar <code>run()</code> en paralelo
<code>run()</code>	Cuerpo del hilo. Llamarlo directo NO crea hilo
<code>join()</code>	Bloquea al llamador hasta que el hilo termine
<code>Thread.sleep(ms)</code>	Pone al hilo actual en <code>TIMED_WAITING</code>
<code>Thread.yield()</code>	Sugiere ceder la CPU
<code>interrupt()</code>	Marca el hilo como interrumpido (lo despierta de <code>sleep/wait</code>)
<code>setDaemon(true)</code>	El hilo no impide la terminación del proceso

El ciclo de vida de un hilo Java tiene los estados `NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING` y `TERMINATED`, alineado con el modelo de la teoría (la JVM agrupa “ejecutando” y “listo” en `RUNNABLE`).

7.b Sincronización: `synchronized` y `wait/notify`

Java provee dos primitivas integradas, sostenidas por una abstracción común: el *monitor*. Cada objeto Java lleva implícitamente un monitor (mutex + variable de condición) que se activa con la palabra clave `synchronized`.

```

1 // Bloque sincronizado: el lock es el objeto entre parentesis.
2 synchronized (obj) { /* region critica */ }
3
4 // Metodo de instancia sincronizado: lock implicito = this.
5 public synchronized void m() { /* ... */ }
6
7 // Metodo estatico sincronizado: lock implicito = Class del tipo.
8 public static synchronized void m() { /* ... */ }

```

Los locks son *reentrant* y se liberan automáticamente al salir del bloque, incluso por excepción.

wait/notify/notifyAll. Solo pueden invocarse mientras se posee el monitor del objeto. `obj.wait()` libera atómicamente el monitor de `obj` y duerme al hilo. `obj.notify()` despierta a uno (cualquiera) de los hilos esperando; `obj.notifyAll()`, a todos.

La regla canónica es esperar siempre dentro de un `while` (no de un `if`) por *spurious wakeups* y por el patrón multi-productor/multi-consumidor:

```

1 synchronized (obj) {
2     while (!condicion) obj.wait();
3     // ahora la condicion es cierta y tengo el monitor
4 }

```

Clasificación. Las primitivas se agrupan en dos familias clásicas: exclusión mutua y sincronización por condición. Java cubre ambas:

Mecanismo Java	Categoría	Equivalente POSIX
<code>synchronized</code>	Exclusión mutua (mutex/monitor)	<code>pthread_mutex_t</code> / <code>flock</code>
<code>wait/notify</code>	Variable de condición	<code>pthread_cond_wait/cond_signal</code>
<code>Thread.join()</code>	Sincronización por finalización	<code>pthread_join</code> / <code>wait()</code> syscall
<code>volatile</code>	Visibilidad entre threads	<code>__atomic_*</code> / barreras de memoria

Además, el paquete `java.util.concurrent` ofrece `ReentrantLock`, `Semaphore`, `CountDownLatch`, `CyclicBarrier`, `BlockingQueue`, etc.

7.c Equivalente Java sin race conditions

```

1 public class CounterExample {
2     private static int counter = 0;
3     private static final Object lock = new Object();
4     private static final int N = 100_000;
5
6     public static void main(String[] args) throws InterruptedException {
7         Thread t1 = new Thread(new Incrementer(1));
8         Thread t2 = new Thread(new Incrementer(2));
9
10        t1.start();
11        t2.start();
12
13        t1.join();
14        t2.join();
15
16        System.out.println("Final Counter Value: " + counter);
17    }
18
19    static class Incrementer implements Runnable {
20        private final int tid;
21
22        Incrementer(int tid) { this.tid = tid; }
23
24        @Override
25        public void run() {
26            System.out.println("Thread " + tid + " running...");
27            for (int i = 0; i < N; i++) {
28                synchronized (lock) {
29                    counter++;
30                }
31            }
32        }
33    }
34 }
```

```

javac CounterExample.java
java CounterExample
```

```

Thread 1 running...
Thread 2 running...
Final Counter Value: 200000
```

Por qué funciona: `synchronized (lock)` garantiza **exclusión mutua** (un solo hilo a la vez en el bloque) y **visibilidad** (el modelo de memoria de Java establece la relación *happens-before*

entre `unlock` y el siguiente `lock`). El incremento se vuelve atómico respecto de los otros hilos y el resultado es determinístico.

Conexión con la teoría

Los bloques `synchronized` de Java son la implementación a nivel de lenguaje de los *monitores* descritos por la teoría (Notas 5–6, *Monitores*): cada objeto Java es un monitor con mutex implícito y una variable de condición asociada (`wait/notify`). La equivalencia conceptual es exacta: `synchronized (obj)` corresponde a `acquire(&obj.lock)`; salir del bloque, a `release(&obj.lock)`; `obj.wait()`, a `wait(&obj.cond, &obj.lock)`; `obj.notify()`, a `signal(&obj.cond)`.

9. Ejercicio 8 — Operaciones del kernel ante interrupción del timer

Se pide describir las operaciones del kernel cuando un proceso es interrumpido por el *timer* y ya consumió su *quantum*, hasta retornar de la interrupción, indicando en qué estado el kernel deja al proceso. La descripción sigue el modelo xv6 sobre RISC-V.

Punto de partida

Un proceso *P* está corriendo en modo usuario cuando ocurre una IRQ del timer (CLINT). La IRQ está delegada de M-mode a S-mode y configurada por el kernel al inicializarse.

Paso 1. La CPU atrapa la interrupción (hardware)

1. Verifica que las interrupciones del modo S estén habilitadas (`sstatus.SIE = 1`).
2. Salva en `sepc` la próxima instrucción de *P*.
3. Salva el modo previo en `sstatus.SPP (U)`.
4. Codifica el motivo en `scause` (timer en supervisor).
5. Cambia el modo a supervisor.
6. Salta a `stvec` (que apuntaba a `uservec`).

Paso 2. uservec

Salva todos los registros de propósito general en el `trapframe` del proceso, conmuta a la pagetable del kernel, carga el stack del kernel y salta a `usertrap()`.

Paso 3. usertrap()

Setea `stvec = kernelvec`, salva `sepc` en el `trapframe`, identifica la causa con `devintr()` y, al ser timer, llama a `yield()`.

Paso 4. yield() cambia el estado a RUNNABLE

1. Toma `p->lock`.
2. Cambia `p->state` de `RUNNING` a `RUNNABLE`.
3. Llama `sched()`, que termina haciendo `swtch(&p->context, &cpu->scheduler)`.

A partir de aquí, *P* está **congelado en RUNNABLE**, sin CPU.

Pasos 5–6. El scheduler ejecuta otros procesos y luego vuelve

`scheduler()` elige una tarea `RUNNABLE` *Q* y le da CPU. Más adelante el scheduler vuelve a seleccionar *P* y hace `swtch(&cpu->scheduler, &p->context)`, reanudándolo dentro de `sched()`.

Pasos 7–8. `usertrapret` → `userret` → `sret`

`usertrapret()` prepara el regreso (`stvec`, `sepc`, `sstatus.SPP=0`, configura el `trapframe`). `userret` conmuta a la pagetable del proceso, restaura los registros y ejecuta `sret`, que en una sola operación atómica restaura `pc` desde `sepc`, restaura el modo a U y rehabilita interrupciones. *P* vuelve a ejecutar como si nada hubiese ocurrido.

Estado final del proceso

A lo largo de la rutina hay dos estados clave:

- Mientras *P* está fuera de la CPU (entre los pasos 4 y 6), su estado en el PCB es **RUNNABLE**. Esa es la respuesta directa al ejercicio: el kernel deja a *P* en **RUNNABLE** después de consumir su quantum.
- Cuando finalmente retorna de la interrupción, *P* está nuevamente en **RUNNING**.

La interrupción del timer no *bloquea* al proceso: solo lo desaloja para darle CPU a otro. Por eso el estado correcto es **RUNNABLE** (sigue siendo elegible para ser planificado), no **SLEEPING** (que se reservaría para esperas por eventos como I/O, caso del Ejercicio 9).

Ruta completa, esquemática

```
[user mode, P]
| HW: timer irq -> sepc, SPP, scause, salta a stvec
v
uservec (trampoline.S) <-- salva regs en trapframe
|
usertrap() en trap.c <-- reconoce timer via devintr()
|
yield() -> p->state = RUNNABLE <-- aca el kernel deja a P
|
sched() -> swtch(&p->context, &cpu->scheduler) <-- 1er ctx switch
v
scheduler() elige otro Q
| swtch(&cpu->scheduler, &Q->context)
| ... eventualmente, scheduler vuelve a P ...
v
swtch() reanuda en sched()/yield() de P
v
usertrapret -> userret -> sret <-- vuelve a modo usuario
v
[user mode, P sigue ejecutando]
```

10. Ejercicio 9 — Operaciones del kernel ante read sobre disco

Se pide la misma descripción del Ejercicio 8 pero cuando el proceso ejecuta el *syscall* `read` sobre un archivo en disco (operación lenta) y describir cómo continúa la ejecución.

Punto de partida

```
1 n = read(fd, buf, len);
```

La función de la libc deja los argumentos en registros y dispara **ECALL** (RISC-V) para entrar al kernel.

Paso 1. Trap por syscall (HW + uservec + usertrap)

Identico al Ejercicio 8 pero `scause = 8` (*environment call from U-mode*). `usertrap()` reconoce que es syscall, avanza `sepc` para que la vuelta apunte a la instrucción siguiente al ECALL, habilita interrupciones y llama a `syscall()`.

Paso 2. Despacho a sys_read

`syscall()` consulta `a7` y ejecuta `sys_read()`, que recupera `fd`, `buf`, `len` del trapframe, valida y llama a `fileread()` → `readi()` → `bread(dev, blockno)`.

Paso 3. Cache miss: arranca I/O y se duerme

`bread()` busca el bloque en la *buffer cache*.

- Cache hit: el bloque está en RAM, se sigue copiando datos al usuario sin bloquearse.
- Cache miss: `virtio_disk_rw(bp, 0)` encola la lectura al controlador VirtIO y luego:

```
1 while ((bp->flags & B_VALID) == 0)
2     sleep(bp, &lock);
```

`sleep(chan, lock)` es la primitiva clave:

1. Toma `p->lock`.
2. Libera el lock que protege la condición.
3. Setea `p->chan = bp` (el “canal” sobre el que duerme).
4. Setea `p->state = SLEEPING`.
5. Llama `sched()` → `swtch`.

A partir de aquí *P* está **SLEEPING** en una *wait queue*. Es justamente esto lo que distingue al `read` con I/O del simple agotamiento de quantum: como la operación puede tardar mucho, el kernel **no** deja al proceso **RUNNABLE**; lo bloquea y le cede la CPU a quien la pueda aprovechar.

Paso 4. El scheduler corre otros procesos

`scheduler()` elige un **RUNNABLE** y le da CPU. Mientras tanto, el disco procesa la lectura *en paralelo* a la CPU.

Paso 5. El disco termina y manda IRQ

`devintr()` reconoce la IRQ del disco y llama a `virtio_disk_intr()`, que marca el buffer como **B_VALID** y ejecuta `wakeup(bp)`: para todo proceso con `state == SLEEPING && chan == bp`, le cambia el estado a **RUNNABLE**.

Paso 6. El scheduler vuelve a P

`swtch(&cpu->scheduler, &p->context)` reanuda *P* dentro de `sleep()`. `sleep` retoma el lock, marca `p->state = RUNNING` y retorna al `while` de `bread()`. Como ahora **B_VALID** está seteado, el `while` termina, `bread` retorna el buffer y `readi` copia los bytes al `buf` de usuario con `copyout()`.

Paso 7. Retorno a modo usuario

El valor de retorno (bytes leídos) se escribe en `p->trapframe->a0` y se retorna por la ruta `usertrap` → `usertrapret` → `userret` → `sret`.

Estados por los que pasó P

Etapas	Estado de P
Antes del read	RUNNING
Dentro de <code>sys_read/bread</code> , esperando I/O	SLEEPING
Después del <code>wakeup(bp)</code>	RUNNABLE
Cuando el scheduler lo vuelve a elegir	RUNNING

Conexión con la teoría

La traza expone el complemento exacto del Ejercicio 8 y reproduce el esquema teórico de las primitivas `suspend/wakeup` (Notas 5–6, *Implementación de tareas en el kernel*):

```
1 // suspend current task in wait queue and yield the cpu
2 void suspend(struct wait_queue *wq) {
3     current->state = WAITING;
4     enqueue(wq, current);
5     yield();
6 }
7
8 // wakeup a process waiting in wait queue
9 void wakeup(struct wait_queue *wq) {
10     struct task *task = dequeue(wq);
11     if (task) task->state = RUNNABLE;
12 }
```

La diferencia clave con el ejercicio anterior: el proceso no es desalojado por *quantum*; se bloquea voluntariamente porque no puede progresar hasta que llegue el dato. Por eso la transición es a SLEEPING y no a RUNNABLE. La cooperación kernel + dispositivo + scheduler es lo que permite la concurrencia entre I/O y CPU.

11. Ejercicio 10 — Demostración semi-formal de Peterson

Se pide mostrar de manera semi-formal que la solución de Peterson (con dos procesos) garantiza *exclusión mutua* y *progreso*.

El algoritmo

```
1 // Variables compartidas, inicializadas en falso / 0
2 bool flag[2] = { false, false };
3 int turn = 0; // o 1, da igual
4
5 // Código que ejecuta el proceso i (j = 1 - i es el otro)
6 void p_i(void) {
7     while (true) {
8         flag[i] = true; // (1) anuncio interes
9         turn = j; // (2) cedo el turno al otro
10        while (flag[j] && turn == j) // (3) busy wait
11            ;
12        // ----- seccion critica -----
13        flag[i] = false; // (4) salgo de la SC
14        // ----- seccion no critica -----
15    }
16 }
```

Hipótesis del modelo.

- Las lecturas y escrituras a `flag[i]` y a `turn` son atómicas.
- Solo P_i escribe en `flag[i]`; `turn` la escriben ambos.
- No se asume orden estricto de ejecución entre P_0 y P_1 : el scheduler puede intercalar arbitrariamente.

Parte A. Exclusión mutua

Afirmación. En ningún estado alcanzable del sistema, ambos procesos pueden estar simultáneamente dentro de la sección crítica.

Demostración por contradicción. Supongamos que en algún instante t , P_0 y P_1 están ambos en la SC. Justo antes de entrar:

- P_0 salió de su `while` (3), por lo que la guarda falló:

$$\neg(\text{flag}[1] = \text{true} \wedge \text{turn} = 1) \Rightarrow \text{flag}[1] = \text{false} \vee \text{turn} = 0 \quad (\star)$$

- Simétricamente para P_1 :

$$\text{flag}[0] = \text{false} \vee \text{turn} = 1 \quad (\star\star)$$

Como ambos están *dentro* de la SC, ambos ya ejecutaron su paso (1):

$$\text{flag}[0] = \text{true} \quad \text{y} \quad \text{flag}[1] = \text{true}.$$

(Ninguno limpia su flag hasta el paso (4), que está después de la SC.)

Reemplazando en $(\star)$ y $(\star\star)$:

- De $(\star)$: como `flag[1] = true`, debe ser `turn = 0`.
- De $(\star\star)$: como `flag[0] = true`, debe ser `turn = 1`.

Por lo tanto `turn = 0` y `turn = 1` simultáneamente. Contradicción. ■

¿Por qué el orden $(1) \rightarrow (2)$ es esencial? Si se invirtieran (primero `turn = j`, después `flag[i] = true`), la prueba se rompe: un proceso podría leer la flag del otro antes de que el otro haya anunciado interés, y ambos podrían colarse simultáneamente.

Parte B. Progreso

Afirmación. Si la SC está libre y al menos un proceso quiere entrar, alguno entrará en tiempo finito. El único proceso que puede impedirle el ingreso a otro es uno que también quiere entrar.

Caso 1: P_0 quiere entrar y P_1 está en la sección no crítica. P_1 no está en (1)–(3): por lo tanto `flag[1] = false`. La guarda del `while` de P_0 evalúa `flag[1] ∧ turn = 1 ⇒ false`. P_0 entra. P_1 no influye.

Caso 2: ambos procesos quieren entrar. Ambos ejecutaron (1), ambos `flag` quedaron en `true`. Sea cual sea el valor final de `turn` antes de que alguno entre al `while`:

- Si `turn = 1`: la guarda de P_0 es verdadera (gira), la de P_1 es falsa. P_1 **entra**.
- Si `turn = 0`: simétricamente, P_0 **entra**.

En cualquier caso hay un ganador. Como `turn` ya no se vuelve a tocar mientras quien tiene el turno esté esperando o dentro de la SC, el ganador no es derrotado por nuevas escrituras.

Caso 3: el proceso que estaba en la SC ya terminó. Al salir ejecutó (4) (`flag[i] = false`); la guarda del otro ve `flag[i] = false`, falla, y entra. Ningún proceso queda atrapado por uno que ya terminó la SC. ■

Espera acotada (bonus)

Para dos procesos la cota es trivial: a lo sumo el otro proceso entra *una sola vez* entre que yo expreso interés y entro yo. Cuando P_j sale de la SC y vuelve a competir, ejecuta nuevamente (2) con `turn = i`; a partir de ahí mi `while` falla y entro yo.

Hipótesis críticas y limites

1. **Atomicidad** de las lecturas/escrituras a `flag[]` y `turn`.
2. **Modelo de memoria sequentially consistent**. Si la máquina puede reordenar las escrituras (1) y (2) —lo que pasa en x86/ARM/RISC-V con buffers de escritura— el algoritmo es semi-formalmente correcto pero prácticamente roto. Una implementación real de Peterson requiere *barreras de memoria* (MFENCE en x86, `fence rw,rw` en RISC-V) entre los pasos (1) y (2).

Por eso, en la práctica moderna, los kernels usan *spinlocks* contruidos sobre instrucciones atómicas del hardware (TAS, CAS, FAA) en lugar de Peterson. Peterson queda como demostración de existencia y pieza pedagógica fundamental.

12. Ejercicio 11 — Productor-consumidor con semáforos POSIX

Se pide implementar el problema del productor-consumidor con la API de semáforos POSIX en dos variantes:

1. Usando POSIX threads.
2. Usando procesos y *named semaphores*, con buffer compartido (archivo o shared memory POSIX).

Arquitectura común

Solución clásica de Dijkstra con buffer acotado y tres semáforos:

Semáforo	Significado	Init	Lo decrementa	Lo incrementa
<code>empty</code>	slots libres	<code>BUF_SIZE</code>	Productor antes de escribir	Consumidor tras leer
<code>full</code>	slots ocupados	0	Consumidor antes de leer	Productor tras escribir
<code>mtx</code>	mutex sobre el buffer	1	Quien va a tocar el buffer	Quien lo libera

```
Productor Consumidor
-----
sem_wait(empty) sem_wait(full)
sem_wait(mtx) sem_wait(mtx)
    buffer[in] = item item = buffer[out]
    in = (in+1) % BUF_SIZE out = (out+1) % BUF_SIZE
sem_post(mtx) sem_post(mtx)
sem_post(full) sem_post(empty)
```

Justificación del orden de los wait.

- `sem_wait(empty)` antes que `sem_wait(mtx)`. Si fuera al revés, el productor podría tomar el mutex teniendo el buffer lleno y dejar al consumidor sin chance de avanzar: *deadlock*.
- El mutex se mantiene **sólo** para la actualización del buffer y de los índices. El *trabajo de producir/consumir* queda fuera para no serializar todo el pipeline.
- `empty` y `full` actúan como variables de condición: bloquean al productor cuando el buffer está lleno y al consumidor cuando está vacío.

11.1 Versión con POSIX threads

Como los hilos comparten memoria, alcanza con declarar el buffer y los semáforos como variables globales del proceso. Los semáforos son *sin nombre* (`sem_t` con `sem_init`).

```
1 #include <pthread.h>
2 #include <semaphore.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6
7 #define BUF_SIZE 8
8 #define N_ITEMS 32
9
10 static int buffer[BUF_SIZE];
11 static int in_idx = 0;
12 static int out_idx = 0;
13
14 static sem_t empty, full, mtx;
15
16 static void *producer(void *arg) {
17     (void)arg;
18     for (int i = 0; i < N_ITEMS; i++) {
19         int item = i * i;
20
21         sem_wait(&empty);
22         sem_wait(&mtx);
23         buffer[in_idx] = item;
24         in_idx = (in_idx + 1) % BUF_SIZE;
25         printf("[P] produjo %d (in=%d)\n", item, in_idx);
26         sem_post(&mtx);
27         sem_post(&full);
28
29         usleep(10 * 1000);
30     }
31     return NULL;
32 }
33
34 static void *consumer(void *arg) {
35     (void)arg;
36     for (int i = 0; i < N_ITEMS; i++) {
37         sem_wait(&full);
38         sem_wait(&mtx);
39         int item = buffer[out_idx];
40         out_idx = (out_idx + 1) % BUF_SIZE;
41         printf("[C] consumo %d (out=%d)\n", item, out_idx);
42         sem_post(&mtx);
43         sem_post(&empty);
44
45         usleep(25 * 1000);
46     }
47     return NULL;
48 }
49
50 int main(void) {
51     sem_init(&empty, 0, BUF_SIZE);
52     sem_init(&full, 0, 0);
53     sem_init(&mtx, 0, 1);
54
55     pthread_t tp, tc;
56     pthread_create(&tp, NULL, producer, NULL);
57     pthread_create(&tc, NULL, consumer, NULL);
58     pthread_join(tp, NULL);
```

```

59     pthread_join(tc, NULL);
60
61     sem_destroy(&empty); sem_destroy(&full); sem_destroy(&mtx);
62     return 0;
63 }

```

```

cc -O2 -Wall -Wextra -pthread -o prod_cons_threads prod_cons_threads.c
./prod_cons_threads

```

```

[P] produjo 0 (in=1)
[P] produjo 1 (in=2)
[C] consumo 0 (out=1)
[P] produjo 4 (in=3)
[C] consumo 1 (out=2)
[P] produjo 9 (in=4)
...

```

El productor adelanta al consumidor hasta que se llena el buffer (`empty` se vuelve 0); ahí se bloquea en `sem_wait(empty)` hasta que el consumidor libere un slot.

Nota. En macOS/Darwin `sem_init()` está deprecado y devuelve `ENOSYS` en runtime; este programa está pensado para GNU/Linux. Para macOS, ver la versión 11.2 con *named semaphores*.

11.2 Versión con procesos y *named semaphores*

Los procesos no comparten memoria por defecto, así que hace falta:

1. Una región de **shared memory POSIX** (`shm_open` + `ftruncate` + `mmap`) para el buffer y los índices.
2. **Semáforos con nombre** (`sem_open`) que vivan en el namespace global del kernel.

```

1  #include <fcntl.h>
2  #include <semaphore.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5  #include <string.h>
6  #include <sys/mman.h>
7  #include <sys/stat.h>
8  #include <sys/wait.h>
9  #include <unistd.h>
10
11 #define BUF_SIZE 8
12 #define N_ITEMS 32
13
14 #define NAME_EMPTY "/so_pc_empty"
15 #define NAME_FULL "/so_pc_full"
16 #define NAME_MTX "/so_pc_mtx"
17 #define NAME_SHM "/so_pc_shm"
18
19 typedef struct {
20     int buffer[BUF_SIZE];
21     int in_idx;
22     int out_idx;
23 } shared_t;
24
25 static void die(const char *m) { perror(m); exit(1); }
26
27 int main(void) {

```

```

28 sem_unlink(NAME_EMPTY);
29 sem_unlink(NAME_FULL);
30 sem_unlink(NAME_MTX);
31 shm_unlink(NAME_SHM);
32
33 int fd = shm_open(NAME_SHM, O_CREAT | O_RDWR, 0600);
34 if (fd < 0) die("shm_open");
35 if (ftruncate(fd, sizeof(shared_t)) != 0) die("ftruncate");
36
37 shared_t *sh = mmap(NULL, sizeof(shared_t),
38                     PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
39 if (sh == MAP_FAILED) die("mmap");
40 memset(sh, 0, sizeof(*sh));
41
42 sem_t *empty = sem_open(NAME_EMPTY, O_CREAT | O_EXCL, 0600, BUF_SIZE);
43 sem_t *full = sem_open(NAME_FULL, O_CREAT | O_EXCL, 0600, 0);
44 sem_t *mtx = sem_open(NAME_MTX, O_CREAT | O_EXCL, 0600, 1);
45 if (empty==SEM_FAILED || full==SEM_FAILED || mtx==SEM_FAILED)
46     die("sem_open");
47
48 pid_t pid = fork();
49 if (pid < 0) die("fork");
50
51 if (pid == 0) {
52     for (int i = 0; i < N_ITEMS; i++) {
53         sem_wait(full);
54         sem_wait(mtx);
55         int item = sh->buffer[sh->out_idx];
56         sh->out_idx = (sh->out_idx + 1) % BUF_SIZE;
57         printf(" [C pid=%d] consumo %d (out=%d)\n",
58              (int)getpid(), item, sh->out_idx);
59         sem_post(mtx);
60         sem_post(empty);
61         usleep(25 * 1000);
62     }
63     exit(0);
64 }
65
66 for (int i = 0; i < N_ITEMS; i++) {
67     int item = i * i;
68     sem_wait(empty);
69     sem_wait(mtx);
70     sh->buffer[sh->in_idx] = item;
71     sh->in_idx = (sh->in_idx + 1) % BUF_SIZE;
72     printf("[P pid=%d] produjo %d (in=%d)\n",
73          (int)getpid(), item, sh->in_idx);
74     sem_post(mtx);
75     sem_post(full);
76     usleep(10 * 1000);
77 }
78
79 waitpid(pid, NULL, 0);
80
81 sem_close(empty); sem_close(full); sem_close(mtx);
82 sem_unlink(NAME_EMPTY);
83 sem_unlink(NAME_FULL);
84 sem_unlink(NAME_MTX);
85 munmap(sh, sizeof(*sh));
86 close(fd);
87 shm_unlink(NAME_SHM);
88 return 0;
89 }

```

Compilación:

```
# Linux: hace falta -lrt para shm_open
cc -O2 -Wall -Wextra -o prod_cons_processes prod_cons_processes.c -lrt
# macOS: shm_open/sem_open viven en libc, no se necesita -lrt
cc -O2 -Wall -Wextra -o prod_cons_processes prod_cons_processes.c
```

```
[P pid=12345] produjo 0 (in=1)
  [C pid=12346] consumo 0 (out=1)
[P pid=12345] produjo 1 (in=2)
[P pid=12345] produjo 4 (in=3)
  [C pid=12346] consumo 1 (out=2)
...
```

pid distinto en cada línea: confirma que productor y consumidor son *procesos separados* que sin embargo comparten el mismo *buffer* y los mismos semáforos.

Por qué `O_EXCL` y `sem_unlink`. `sem_open` con `O_CREAT | O_EXCL` falla si el semáforo ya existe; eso obliga a una creación limpia y avisa si una corrida anterior dejó basura en el sistema.

Por qué *shared memory* y no *archivo*. Es más rápido (no toca el filesystem); `mmap` `MAP_SHARED` sobre el fd de `shm_open` pone una sola estructura en RAM accesible a ambos procesos sin sincronización adicional más allá del mutex POSIX. En sistemas modernos las regiones aparecen en `/dev/shm/<nombre>` y son fáciles de inspeccionar durante el desarrollo.

Por qué la solución no tiene los problemas clásicos

- **Sin lost wakeup:** los semáforos contadores recuerdan los `post()` aunque no haya nadie esperando. Si el consumidor llega antes que el productor, `sem_wait(full)` lo deja dormido; cuando el productor hace `sem_post(full)`, el consumidor se despierta.
- **Sin deadlock:** ambos toman primero los contadores y después el mutex; el mutex se libera en cada iteración. No hay ciclos.
- **Sin starvation:** `sem_wait` en POSIX es FIFO en la mayoría de las implementaciones (no es estrictamente requerido por el estándar pero es lo habitual en glibc/Linux).
- **Sin race condition** sobre `in_idx/ out_idx`: ambos viven dentro de la región crítica.

Conexión con la teoría

El algoritmo es exactamente el del Listado 8 de la teoría (*Concurrencia e IPC, Semáforos*). El ejercicio cubre los *dos modelos* de concurrencia descritos en clase:

1. **Memoria compartida intra-proceso** (versión con threads): los hilos comparten todas las variables globales naturalmente.
2. **Shared memory POSIX entre procesos** (versión con procesos): la región se crea explícitamente con `shm_open + mmap`; los semáforos requieren ser *named* para ser visibles entre namespaces de procesos distintos. Esto corresponde al mecanismo IPC *Shared memory* de la Tabla 1 de las Notas (*IPC en UNIX*).

La elección de tres semáforos sigue el patrón clásico: dos contadores para sincronización por condición (replazan al `wait/signal` de las variables de condición) más un mutex binario para exclusión mutua sobre el buffer.

13. Ejercicio 12 — Deadlock entre `interrupt_handler` y `f()`

Enunciado. Se tiene el siguiente fragmento de un SO en el que una función `f()` y un manejador de interrupción `interrupt_handler()` comparten un recurso `r` protegido por un *spinlock* `l`:

```
1 lock l = 0;
2 resource r;
3
4 void interrupt_handler() {
5     acquire(&l);
6     use_resource(r);
7     release(&l);
8 }
9
10 void f() {
11     // do something...
12     acquire(&l);
13     use_resource(&l);
14     release(&l);
15 }
16
17 void acquire(lock *l) {
18     while (atomic_swap(l, 1) == 1)
19         ; // spin hasta tomar el lock
20 }
21
22 void release(lock *l) {
23     *l = 0;
24 }
```

Se pide: (A) dar una traza que muestre que puede ocurrir *deadlock*; (B) discutir si puede considerarse *livelock*; (C) proponer una corrección de `acquire/release` para evitarlo.

Análisis

Hay un detalle que cambia todo: `interrupt_handler()` no es una función ordinaria. Se ejecuta **en el mismo flujo de la CPU** que estaba interrumpiendo, asincrónicamente, cuando llega la IRQ del dispositivo. Es decir: si `f()` estaba corriendo y *ya había tomado el lock*, la CPU pasa a `interrupt_handler()` *con el lock tomado por el mismo flujo*, y el handler intenta tomarlo de nuevo. El `atomic_swap` ve `l = 1`, entra al `while` y **espera por un lock que solo él podría liberar**.

Esa es la causa estructural: en un solo procesador, un *spinlock* *no es seguro* para sincronizar entre un flujo normal y un handler de interrupción si el handler corre *con interrupciones habilitadas y sin deshabilitarlas antes de adquirir el lock*. La regla clásica de los kernels (por ejemplo, en Linux) es que cualquier *spinlock* que pueda ser tomado en contexto de interrupción se adquiere con `spin_lock_irqsave()`, que combina la toma del lock con la *inhibición de interrupciones* en la CPU local.

A. Traza que dispara el deadlock

CPU única, una sola entrada en la cola de planificación. Estado inicial: `l = 0`.

```
t0: f() ejecuta acquire(&l)
    -> atomic_swap(l, 1) devuelve 0 -> l = 1, lock tomado por f()
t1: f() comienza use_resource(...)
t2: llega IRQ del dispositivo
    -> la CPU salta a interrupt_handler() SIN liberar el lock
    -> el contexto "f()" queda suspendido pero el lock sigue en 1
```

```

t3: interrupt_handler() ejecuta acquire(&l)
    -> atomic_swap(1, 1) devuelve 1 (l ya valia 1)
    -> entra al while (spin)
t4: spin infinito:
    - el handler no puede progresar (esta girando)
    - f() no puede retomar la CPU (la tiene el handler)
    - nadie va a hacer release(&l)

```

La CPU queda atrapada *indefinidamente* en el spin del handler. Esto es un **deadlock** clásico: hay un ciclo de espera trivial, con un único actor que depende de sí mismo bloqueado por una cesión involuntaria de control.

Las cuatro condiciones de Coffman

Para que haya deadlock se tienen que dar simultáneamente: exclusión mutua, retención y espera (*hold-and-wait*), no apropiación (*no preemption*) y espera circular. Aquí **las cuatro se cumplen**: el lock es de uso exclusivo; `f()` retiene el lock mientras la CPU pasa al handler (espera); el handler no puede “quitarle” el lock a `f()`; y el ciclo es **handler** → lock → `f()` → CPU → **handler**.

B. ¿Es livelock?

No exactamente. El *livelock* se caracteriza porque los actores *siguen consumiendo CPU activamente* ejecutando código, pero ninguno hace progreso útil. En sentido amplio, sí: la CPU está ejecutando una instrucción tras otra (el bucle `while` del `acquire`), no está parada ni en HALT. Si uno mira la máquina, la ve trabajando.

Pero por el criterio clásico de planificación, sigue siendo un **deadlock**: `f()` está bloqueado (no porque esté durmiendo, sino porque le secuestraron la CPU) y el handler está bloqueado esperando un lock que nadie va a liberar. No hay *ningún* estado alcanzable en el que el sistema progrese, que es la definición operativa de deadlock.

En la literatura del curso, el caso suele clasificarse como deadlock con *busy-wait* (el actor activo gasta CPU pero no avanza), distinto al livelock genuino, donde múltiples actores *cambian de estado activamente* pero su interacción los mantiene sin progreso (típicamente dos procesos que ceden el paso mutuamente y no avanzan ninguno).

C. Corrección: deshabilitar interrupciones al adquirir

La corrección estándar es asegurarse de que, mientras se posee un lock que puede ser solicitado por un handler de interrupción, las interrupciones estén *deshabilitadas* en la CPU local. Así, el handler no puede correr en medio de la región crítica y no puede intentar tomar el lock.

```

1  /*
2   * Implementacion correcta para spinlock que puede ser solicitado
3   * tanto desde codigo normal como desde un interrupt handler.
4   */
5
6  void acquire(lock *l) {
7      intr_disable(); // deshabilita IRQs locales
8      while (atomic_swap(1, 1) == 1) {
9          intr_enable(); // habilita brevemente
10         // (asi otras IRQs no se pierden mientras giramos)
11         cpu_relax(); // pausa, hint al HW
12         intr_disable(); // vuelve a deshabilitar
13     }
14     /* en este punto: lock tomado Y IRQs deshabilitadas */
15 }

```

```

16
17 void release(lock *l) {
18     *l = 0;
19     intr_enable(); // las rehabilito
20 }

```

Análisis de la corrección:

- Cuando `f()` llama a `acquire()`, deshabilita las interrupciones de la CPU local. A partir de ese instante el handler *no puede correr* en este núcleo.
- Si la IRQ se dispara mientras `f()` tiene el lock, el hardware la deja *pendiente*; será atendida en cuanto `release` rehabilite las interrupciones.
- En el caso simétrico (el handler tomando el lock en un sistema multicore donde `f()` corre en otro núcleo), el spin sigue siendo correcto: la otra CPU eventualmente liberará.
- Las dos líneas `intr_enable/intr_disable` dentro del bucle son una optimización: en multiprocesador, no quiero perder IRQs de *otros* dispositivos mientras espero.

En Linux: `spin_lock_irqsave`

La API de Linux para este patrón es `spin_lock_irqsave(&l, flags)` (guarda el estado de IRQ previo) y `spin_unlock_irqrestore(&l, flags)` (lo restaura). Es exactamente lo mismo que la corrección de arriba, pero preservando si las IRQs ya estaban deshabilitadas (caso anidado). En multiprocesador, los spinlocks de Linux son *de a uno por CPU* y bloquean en la CPU local mientras el spin se hace por el lock global.

Conclusión. El bug original es que `f()` adquiere el lock con interrupciones habilitadas, y luego una interrupción puede “aparecer” en medio de la región crítica y pedir el mismo lock. La corrección es trivial pero esencial: *cualquier* lock que pueda ser solicitado desde un manejador de interrupción debe adquirirse con las IRQs locales deshabilitadas.

14. Ejercicio 13 — Grafo de recursos y deadlock con liberación temprana

Enunciado. Dado el siguiente uso de tres procesos y tres recursos:

1. P_1 adquiere R_1 .
2. P_2 adquiere R_2 .
3. P_3 adquiere R_3 .
4. P_1 requiere R_2 .
5. P_2 requiere R_3 .
6. P_3 requiere R_1 .

Dibujar el grafo de *uso* (asignación) y *requerimiento* de recursos e indicar si hay *deadlock*. Después analizar qué pasa si P_1 libera R_1 *antes* del paso 4.

Análisis

El **grafo de asignación de recursos** es un grafo dirigido bipartito con dos tipos de nodos: procesos (círculo) y recursos (cuadrado). Las aristas se interpretan así:

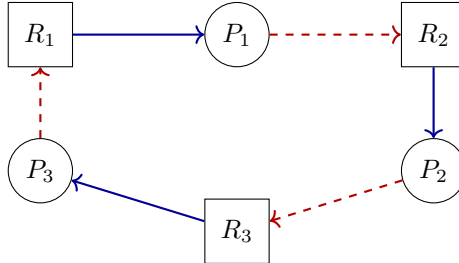
- $R \rightarrow P$ (de recurso a proceso): el recurso *está asignado* a ese proceso.
- $P \rightarrow R$ (de proceso a recurso): el proceso *está esperando* ese recurso.

Criterio. Si todos los recursos son de *instancia única*, hay deadlock $\iff$ existe un ciclo en el grafo.

Caso 1: el escenario completo

Después del paso 6:

- Aristas de asignación: $R_1 \rightarrow P_1$, $R_2 \rightarrow P_2$, $R_3 \rightarrow P_3$.
- Aristas de petición: $P_1 \rightarrow R_2$, $P_2 \rightarrow R_3$, $P_3 \rightarrow R_1$.



(Las aristas azules sólidas son asignaciones; las rojas punteadas son pedidos pendientes.)

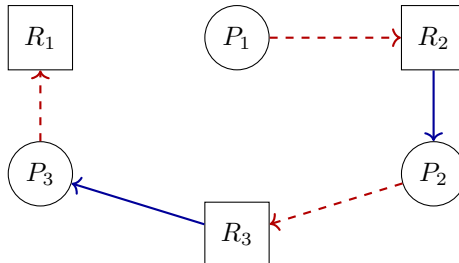
Siguiendo el ciclo: $P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$.

Conclusión del caso 1: hay deadlock. Existe un ciclo cerrado: cada proceso espera un recurso retenido por el siguiente, formando un *ciclo de espera* de longitud 3. Como los recursos son únicos, el ciclo basta para garantizar deadlock; las cuatro condiciones de Coffman se cumplen.

Caso 2: P_1 libera R_1 antes del paso 4

Si entre los pasos 3 y 4 ocurre el evento adicional “ P_1 libera R_1 ”, el grafo queda así al llegar al paso 6:

- Aristas de asignación: $R_2 \rightarrow P_2$, $R_3 \rightarrow P_3$. R_1 está libre.
- Aristas de petición: $P_1 \rightarrow R_2$, $P_2 \rightarrow R_3$, $P_3 \rightarrow R_1$.



No hay ciclo. La arista $R_1 \rightarrow P_1$ desapareció; R_1 no tiene aristas entrantes ni salientes. Toda cadena de espera termina en algún recurso libre (en concreto, R_1).

Progreso de los procesos hasta su finalización

Como hay un grafo acíclico de espera, existe al menos un orden válido de finalización (es lo que enuncia el *algoritmo de detección estándar*). Veamos cómo evolucionan los procesos:

1. P_3 **obtiene** R_1 . R_1 estaba libre y P_3 lo pedía; el SO se lo asigna. Ahora P_3 tiene R_3 y R_1 .
2. P_3 **termina su trabajo y libera** R_3 y R_1 .
3. P_2 **obtiene** R_3 . Lo estaba esperando; ahora tiene R_2 y R_3 .
4. P_2 **termina y libera** R_2 y R_3 .
5. P_1 **obtiene** R_2 . Termina y libera todo.

Diagrama temporal de la progresión:

```

estado inicial (tras la liberacion temprana de R1):
  R1 libre R2 -> P2 R3 -> P3
  P1 espera R2 P2 espera R3 P3 espera R1

paso 1: P3 toma R1 (estaba libre)
  R1 -> P3 R2 -> P2 R3 -> P3
  P1 espera R2 P2 espera R3 P3 listo para correr

paso 2: P3 termina, libera R3 y R1
  R1 libre R2 -> P2 R3 libre

paso 3: P2 toma R3
  R1 libre R2 -> P2 R3 -> P2
  P1 espera R2 P2 listo para correr

paso 4: P2 termina, libera R2 y R3
  R1 libre R2 libre R3 libre

paso 5: P1 toma R2
  R2 -> P1
  P1 corre, termina, libera R2

estado final: todos los recursos libres, todos los procesos terminaron.

```

Conclusión.

- En el escenario original (pasos 1–6 sin liberación) **sí hay deadlock**: el grafo de asignación tiene un ciclo de longitud 3.
- Si P_1 libera R_1 antes del paso 4, **no hay deadlock**: el ciclo se rompe en su origen y existe una secuencia de finalización ($P_3 \rightarrow P_2 \rightarrow P_1$) que lleva a todos los procesos a terminar.

Esta es la base del **algoritmo del banquero** y de las técnicas de *detección* y *prevención* de deadlocks: si en cada estado del grafo de asignación de recursos existe al menos una secuencia segura de finalización, el sistema está libre de deadlock.

15. Ejercicio 14 — Productor-consumidor con mq POSIX y señales

Enunciado. Implementar el problema productor-consumidor usando **procesos** y **colas de mensajes POSIX** (`mq_open`, `mq_send`, `mq_receive`). El productor debe bloquearse cuando la cola está llena; el consumidor, cuando está vacía. La sincronización debe lograrse usando las señales `SIGSTOP` y `SIGCONT`: un proceso se auto-bloquea enviándose `SIGSTOP` a sí mismo y es despertado por el otro proceso mediante `SIGCONT`. El formato de los mensajes lo elegimos a conveniencia.

Análisis

Las **colas de mensajes POSIX** son un mecanismo IPC con namespace global del kernel (los descriptores se nombran como `/nombre`). Cada cola tiene un tamaño máximo de mensajes (`mq_maxmsg`) y un tamaño máximo por mensaje (`mq_msgsize`). Por defecto, `mq_send` se bloquea si la cola está llena y `mq_receive` se bloquea si está vacía.

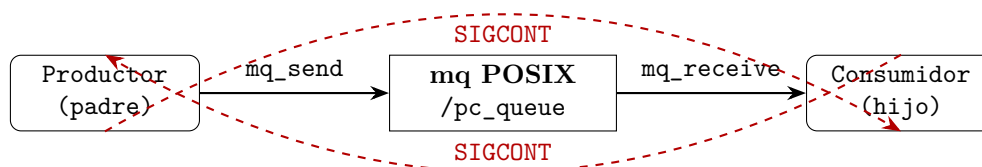
¿Por qué entonces hace falta `SIGSTOP/SIGCONT`?

Justamente, si dejáramos que las funciones `mq_*` se bloqueen por sí solas, **el ejercicio sería trivial** y no enseñaría nada sobre control explícito de procesos. La consigna pide

nosotros detectar las condiciones de cola llena/vacía con la flag `O_NONBLOCK` y, en esos casos, suspender el proceso con `kill(getpid(), SIGSTOP)`. El otro proceso, al ver que pudo publicar/consumir un mensaje, lo despierta con `kill(otro_pid, SIGCONT)`. Es un ejercicio didáctico: muestra que las señales pueden implementar primitivas de sincronización (es lo que hizo históricamente `cu` con `SIGTTIN/SIGTTOU`, y lo que hacen los debuggers).

Arquitectura

- Un proceso **padre** crea la cola (`mq_open(O_CREAT|O_RDWR|O_NONBLOCK)`) e intercambia los PIDs con el hijo a través de un *pipe* (necesario porque el productor necesita saber el PID del consumidor y viceversa).
- `fork()`: el hijo es **consumidor**; el padre es **productor**.
- Formato del mensaje: una estructura simple con un número de secuencia y una carga útil de texto.
- Cola no bloqueante: si `mq_send` retorna con `errno == EAGAIN`, el productor se auto-suspende. Análogamente `mq_receive` para el consumidor.



Solución

```

1  /* prod_cons_mq.c -- compilar con: cc -O2 -Wall -Wextra prod_cons_mq.c -lrt */
2  #define _POSIX_C_SOURCE 200809L
3  #include <errno.h>
4  #include <fcntl.h>
5  #include <mqueue.h>
6  #include <signal.h>
7  #include <stdio.h>
8  #include <stdlib.h>
9  #include <string.h>
10 #include <sys/wait.h>
11 #include <unistd.h>
12
13 #define MQ_NAME "/pc_queue_demo"
14 #define MAX_MSG 4 /* capacidad de la cola */
15 #define MSG_SIZE sizeof(msg_t)
16 #define N_ITEMS 16 /* cuantos items produce el productor */
17
18 typedef struct {
19     int seq;
20     char payload[24];
21 } msg_t;
22
23 static void die(const char *m) { perror(m); exit(1); }
24
25 /* SIGCONT solo despierta; no necesita hacer nada en el handler. */
26 static void noop_handler(int sig) { (void)sig; }
27
28 int main(void) {

```

```

29  /* Limpieza por si quedo basura de una corrida previa. */
30  mq_unlink(MQ_NAME);
31
32  /* Atributos de la cola: no-bloqueante, capacidad acotada. */
33  struct mq_attr attr = {
34      .mq_flags = O_NONBLOCK,
35      .mq_maxmsg = MAX_MSG,
36      .mq_msgsize = MSG_SIZE,
37      .mq_curmsgs = 0,
38  };
39
40  mqd_t mq = mq_open(MQ_NAME,
41                    O_CREAT | O_RDWR | O_NONBLOCK,
42                    0600, &attr);
43  if (mq == (mqd_t)-1) die("mq_open");
44
45  /* Instalo handler de SIGCONT (default es ignore + wake up). */
46  struct sigaction sa = {0};
47  sa.sa_handler = noop_handler;
48  sigemptyset(&sa.sa_mask);
49  sigaction(SIGCONT, &sa, NULL);
50
51  /* Necesitamos que cada proceso conozca al otro PID. */
52  int pipe_p2c[2], pipe_c2p[2];
53  if (pipe(pipe_p2c) < 0 || pipe(pipe_c2p) < 0) die("pipe");
54
55  pid_t pid = fork();
56  if (pid < 0) die("fork");
57
58  if (pid == 0) {
59      /* ----- HIJO: CONSUMIDOR ----- */
60      close(pipe_p2c[1]);
61      close(pipe_c2p[0]);
62      pid_t prod_pid;
63      if (read(pipe_p2c[0], &prod_pid, sizeof(prod_pid)) <= 0)
64          die("read prod_pid");
65      pid_t self = getpid();
66      write(pipe_c2p[1], &self, sizeof(self));
67
68      for (int i = 0; i < N_ITEMS; i++) {
69          msg_t m;
70          unsigned prio;
71
72          /* Intentar recibir; si la cola esta vacia, auto-suspender. */
73          while (mq_receive(mq, (char *)&m, MSG_SIZE, &prio) < 0) {
74              if (errno == EAGAIN) {
75                  printf(" [C] cola vacia, me suspendo\n");
76                  fflush(stdout);
77                  kill(self, SIGSTOP); /* duermo hasta SIGCONT */
78                  continue; /* y reintento mq_receive */
79              }
80              die("mq_receive");
81          }
82          printf(" [C] consumo seq=%d payload=\"%s\"\n",
83                m.seq, m.payload);
84          fflush(stdout);
85
86          /* Avisar al productor por si se durmio por cola llena. */
87          kill(prod_pid, SIGCONT);
88
89          usleep(80 * 1000); /* consumo lento */
90      }
91  }

```

```

92     close(pipe_p2c[0]); close(pipe_c2p[1]);
93     mq_close(mq);
94     exit(0);
95 }
96
97 /* ----- PADRE: PRODUTOR ----- */
98 close(pipe_p2c[0]);
99 close(pipe_c2p[1]);
100 pid_t self = getpid();
101 write(pipe_p2c[1], &self, sizeof(self));
102 pid_t cons_pid;
103 if (read(pipe_c2p[0], &cons_pid, sizeof(cons_pid)) <= 0)
104     die("read cons_pid");
105
106 for (int i = 0; i < N_ITEMS; i++) {
107     msg_t m = { .seq = i };
108     snprintf(m.payload, sizeof(m.payload), "item-%d", i);
109
110     /* Intentar enviar; si la cola esta llena, auto-suspender. */
111     while (mq_send(mq, (const char *)&m, MSG_SIZE, 0) < 0) {
112         if (errno == EAGAIN) {
113             printf("[P] cola llena, me suspendo\n");
114             fflush(stdout);
115             kill(self, SIGSTOP); /* duermo hasta SIGCONT */
116             continue;
117         }
118         die("mq_send");
119     }
120     printf("[P] produjo seq=%d\n", m.seq);
121     fflush(stdout);
122
123     /* Aviso al consumidor por si esta dormido por cola vacia. */
124     kill(cons_pid, SIGCONT);
125
126     usleep(20 * 1000); /* produccion rapida */
127 }
128
129 waitpid(pid, NULL, 0);
130
131 close(pipe_p2c[1]); close(pipe_c2p[0]);
132 mq_close(mq);
133 mq_unlink(MQ_NAME);
134 return 0;
135 }

```

Compilación y ejecución

```

# Linux: hace falta -lrt para mq_*
cc -O2 -Wall -Wextra -o prod_cons_mq prod_cons_mq.c -lrt
./prod_cons_mq

```

Salida esperada (el productor es más rápido, llena la cola, se suspende; el consumidor lo despierta):

```

[P] produjo seq=0
[P] produjo seq=1
[C] consumo seq=0 payload="item-0"
[P] produjo seq=2
[P] produjo seq=3
[P] produjo seq=4

```



```

[P] cola llena, me suspendo <-- 4 mensajes en cola, SIGSTOP
  [C] consumo seq=1 payload="item-1"
[P] produjo seq=5 <-- SIGCONT del consumidor
[P] produjo seq=6
[P] cola llena, me suspendo
  [C] consumo seq=2 payload="item-2"
[P] produjo seq=7
...
  [C] consumo seq=15 payload="item-15"

```

Por qué funciona

- **El handler de SIGCONT es un no-op.** No hace falta que haga nada: el solo hecho de recibir SIGCONT despierta al proceso de SIGSTOP. La presencia del handler asegura que la señal no se ignore silenciosamente y además *interrumpe* cualquier syscall bloqueante con EINTR (útil si en lugar de O_NONBLOCK se usara una variante bloqueante).
- **La cola actúa como buffer.** POSIX garantiza que mq_send/mq_receive son atómicos respecto a múltiples productores/consumidores, así que no hace falta proteger los índices con un mutex aparte.
- **Las señales solucionan el problema del wake-up perdido.** ¿Y si el consumidor manda SIGCONT *justo* antes de que el productor llame a SIGSTOP? Cuando el productor finalmente se duerma, no habrá nadie que lo despierte. Sin embargo, el bucle **while** re-intenta mq_send después de SIGSTOP, y si la cola dejó de estar llena gracias al consumo previo, el envío tiene éxito sin necesidad de un segundo SIGCONT. La condición se vuelve a evaluar *después* del wakeup.
- **No hay deadlock.** Si productor y consumidor están ambos dormidos, eso significaría cola simultáneamente llena y vacía, lo cual es imposible: MAX_MSG > 0.

Inspección de la cola desde el shell

En Linux las colas de mensajes POSIX se montan opcionalmente como un filesystem virtual:

```

sudo mkdir -p /dev/mqueue
sudo mount -t mqueue none /dev/mqueue
ls -l /dev/mqueue/ # muestra /pc_queue_demo
cat /dev/mqueue/pc_queue_demo # QSIZE: ..., NOTIFY_PID: ...

```

Conclusión

El productor-consumidor con mq POSIX y SIGSTOP/SIGCONT es funcional pero pedagógicamente *barroco*: en producción se preferiría dejar que las syscalls bloqueantes hagan el trabajo (mq_send sin O_NONBLOCK bloquea cuando la cola está llena, sin necesidad de señales). La versión con señales *tiene valor didáctico*: muestra que (1) SIGSTOP es no enmascarable y permite implementar un *sleep* confiable, (2) SIGCONT es la primitiva recíproca, y (3) el patrón “probar, dormir si no hay condición, ser despertado, reintentar” es exactamente la semántica de un *condition variable* con wait/signal: las señales POSIX nos permiten implementar a mano lo que pthread_cond_t implementa por nosotros.

16. Conclusiones

El práctico recorre, de manera práctica y progresiva, los tres pilares del modelo de tareas del SO: la *representación* del proceso (estados, atributos, descriptores expuestos por `ps` y `/proc`), su *planificación* (interrupciones del timer, context switches, prioridades con `nice/renice`) y su *coordinación* ante recursos compartidos (race conditions, `flock`, mutexes POSIX, monitores Java, semáforos, colas de mensajes).

Los dos ejercicios teóricos sobre operaciones del kernel (8 y 9) muestran que la diferencia esencial entre desalojo por *quantum* y bloqueo por I/O es el estado al que se transita: `RUNNABLE` en un caso, `SLEEPING` en el otro. Esa misma distinción —tiempo compartido voluntario vs. espera por evento— es la que justifica toda la arquitectura de las *wait queues*, los semáforos y las variables de condición que se materializan en el productor-consumidor de los Ejercicios 11 y 14.

Peterson (Ejercicio 10) hace el cierre teórico de la exclusión mutua: puede resolverse por software, pero las hipótesis de atomicidad y consistencia secuencial que requiere muestran por qué los sistemas reales recurren a instrucciones atómicas del hardware.

Los Ejercicios 12 y 13 cierran el bloque de *patologías* de la concurrencia: el primero, deadlock causado por una interrupción que intenta tomar un lock ya retenido en el mismo flujo de CPU (y la regla de oro que se desprende: *todo lock pedido desde un handler debe adquirirse con interrupciones deshabilitadas*); el segundo, el clásico ciclo de espera de tres procesos sobre tres recursos, y cómo basta una liberación temprana para romper el grafo y permitir una secuencia segura de finalización.

Todo el práctico converge en una sola idea: la concurrencia es *no determinismo controlado*, y el objetivo del SO es ofrecer abstracciones suficientes para que ese no determinismo no altere los objetivos de cómputo.